

课程笔记

AI Agent Loop 到底是不是神技？Greg Isenberg 和 Ras Mic 聊清楚了

Codex

2026 年 6 月 22 日



视频作者/频道: KrillinAI 小林

发布日期: 2026-06-11

视频时长: 22:32

视频链接: <https://www.bilibili.com/video/BV1D2Ev6YEkn/>

目录

1 Agent Loop 的承诺：从人类在环到代理自循环	1
2 全自动 Loop 的失控点：假设、Token 与产品愿景	4
3 什么时候可以放手：实验、原型与有限目标	8
4 可落地的闭环：Cursor、GitHub 与 Greptile 代码审查	10
5 边界条件：二元反馈、人类反馈与创造性工作	14
6 总结与延伸：最好的 Loop 是有约束的人机协作	18

1 Agent Loop 的承诺：从人类在环到代理自循环

1.1 为什么所有人都在谈论 Agent loop

这段开场要解决一个很基本的问题：Agent loop（代理闭环）到底是什么，为什么它会让 AI 行业里的很多人兴奋，又为什么 Ras Mic 一开始就给出强烈警告。视频里的语气很直接：这个概念真实存在，也确实有实际用法；同时，把它理解成“让代理一直自动做下去”会给普通建造者带来很高的成本和质量风险。

所谓“承诺”，来自一个很诱人的想象：人类只写一次目标、PRD 或 Markdown 规格说明，代理随后自己生成结果、检查结果、修复结果，再继续下一步。对开发者、创业者和产品团队来说，这听起来像把日常提示、检查、返工和沟通压缩成一次启动动作。它把“我持续提示 AI”升级成“我设计一个能自己跑的系统”。

这个承诺之所以有吸引力，是因为它抓住了一个真实痛点。普通的人类在环工作流里，人需要不断检查中间结果：着陆页是否符合定位，认证流程是否能用，后端数据模型是否撑得住，用户看到页面后是否愿意注册。每一步都要看、试、改。Agent loop 试图把这些反馈动作的一部分交给系统自己处理。

这个差异在白板图里很直观：人类在环时，人的提示、测试和批准仍然留在流程中间；代理自循环时，反馈回路更多发生在 AI agent 与结果之间。

1.2 human-in-the-loop：普通建造者熟悉的控制结构

Ras Mic 先画出的第一种模型，是普通建造者已经在使用的 human-in-the-loop（人类在环）。人类给 Cursor、Claude、Codex 或其他 AI 代理下指令，代理生成输出，人类查看、测试、批准或要求修改。这里的“环”并没有消失，区别在于反馈来自人。

可以把一个待办事项应用作为例子。创始人先让代理生成着陆页，因为产品需要对外发布和收集等待名单；着陆页能表达价值主张后，再让代理做认证；认证流程可用后，再推进后端、任务模型、数据库、权限和部署。每个阶段都需要人类确认，因为产品的目标、用户感受和商业节奏会在过程中变清楚。

human-in-the-loop (人类在环)

human-in-the-loop 指人类审查、测试、批准和产品判断被放在工作流内部。它像一个产品经理、测试者和预算负责人同时坐在流程里：产品经理负责方向，测试者负责发现问题，预算负责人负责决定是否继续消耗时间和 Token。

这个控制面让代理可以快速生成和修复，也让人类能在方向偏离时及时接管。对普通建造者来说，人类在环常常是质量控制、成本控制和产品判断的同一个入口。

人类在环的成本很明显：慢，需要人看中间结果，需要人做判断。它的价值也同样明显：错误会比较早暴露，预算消耗可以被及时叫停，产品愿景不会完全交给模型猜测。视频后面所有关于全自动 loop 的批评，都以这个控制结构作为参照。

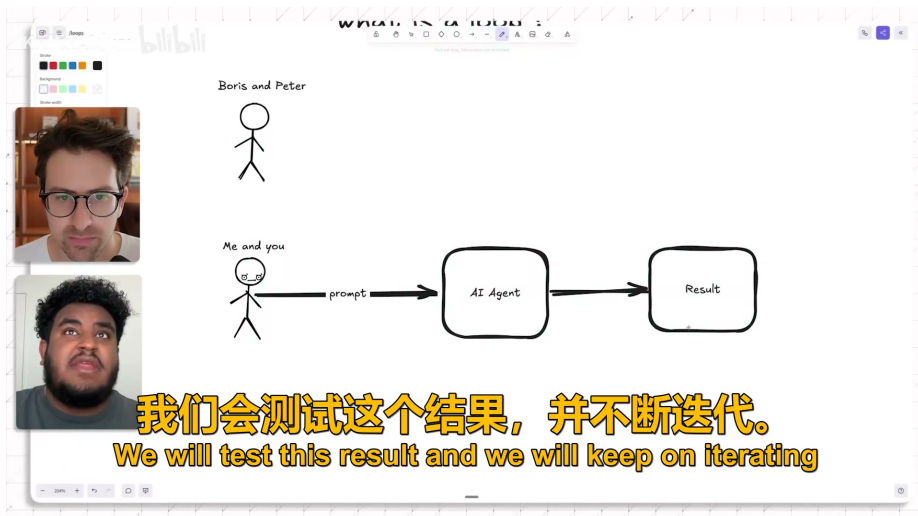


图 1: human-in-the-loop 模式：人类检查结果，并把判断带回下一轮。¹

1.3 代理自循环：从一次触发到连续执行

第二种模型才是视频讨论的主角：Agent loop。人类触发一次，代理开始生成结果；系统对结果进行评估，或者接收某种反馈；这个反馈进入下一轮上下文，代理继续行动。人类参与被压缩到启动、偶尔检查或最终验收，循环内部的大量判断交给代理与反馈机制。

Agent loop (代理闭环) 的工作定义

Agent loop (代理闭环) 是一个最小人类 steering 的自动化工作流：代理先生成输出，再评估输出或接收反馈，然后把这个结果反馈进下一次行动。循环持续运行，直到达到目标、触发停止条件、耗尽预算，或由人类接管。

它的核心组件是：代理行动、反馈信号、下一步行动、停止规则。缺少反馈信号，循环会变成自动猜测；缺少停止规则，循环会持续消耗 Token 和注意力。

这里的关键词是“反馈”。如果代理只是连续生成代码，那只是批量输出；如果代理能读测试结果、审查意见、评分、错误日志或人工评论，并把这些信号变成下一步修改，它才形成真正的闭环。一个闭环的质量，取决于反馈信号是否可靠、范围是否足够小、系统是否知道何时停下。

¹ 视频画面时间区间：00:02:31-00:02:41。

² 视频画面时间区间：00:03:56-00:04:07。

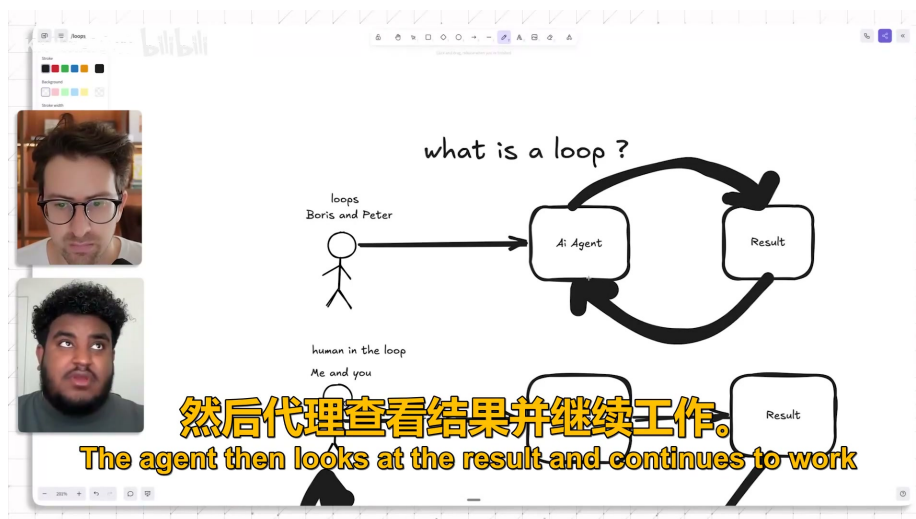


图 2: Agent loop 模式：结果回流到代理，代理继续执行下一轮。²

1.4 为什么 PRD 或 Markdown 规格说明听起来像解决方案

全自动 loop 的想象通常会绑定到 PRD 或 Markdown spec。人类把产品目标、功能需求、页面结构、设计要求和技術约束写进一个文档，然后交给代理执行。表面上看，这像是在给代理提供“所有必要信息”。如果文档足够完整，代理似乎就能从着陆页一路做到认证、后端、部署和修复。

这种想法有一个合理起点：规格说明确实能减少沟通成本。对重复任务、模板任务和实验任务来说，写清楚输入、输出、验收标准和限制条件，会显著提高 AI 代理的可用性。真正的难点藏在“所有必要信息”这几个字里。产品工作里的必要信息往往会在构建过程中出现，初始文档很难一次性覆盖。

Ras Mic 的待办事项应用例子正好说明了这一点。一个页面是否“对”，取决于用户是谁、他们为什么注册、等待名单承诺是否可信、视觉风格是否符合市场、认证流程是否影响转化、后端模型是否服务未来功能。很多判断只有在看到中间版本后才会浮现。PRD 能描述目标，它很难穷尽所有产品取舍。

1.5 承诺背后的真实吸引力

Agent loop 的吸引力可以拆成三层。

- **速度吸引力**：人类减少逐轮提示，代理自己完成更多连续步骤。
- **系统吸引力**：工作从“一个提示得到一个答案”变成“一个机制持续改进输出”。
- **规模吸引力**：只要反馈信号可靠，同一套循环可以重复处理许多相似任务。

这三层吸引力都是真实的。代码审查、测试驱动修复、固定模板内容生成、批量页面生产，都可能从闭环中获益。后续章节的 Greptile 代码审查例子会展示一种更可靠的闭环：Cursor 写代码，GitHub 承载 pull request，Greptile 给出审查和评分，Cursor 根据反馈继续修复。

第 1 章需要先把概念边界立住。Agent loop 的价值来自“反馈能否闭环”，靠代理长时间运行无法保证结果变好。反馈质量低时，代理会把错误假设自动放大；反馈质量高时，代理才可能把局部任务推向可验收状态。

1.6 第一层警告：人类从环里退出后，控制面也会减少

Ras Mic 在开场阶段已经埋下了后续批评：当人类只参与一次启动，代理就必须在大量细节上自行做决定。界面长什么样、功能如何排序、数据模型如何设计、错误状态如何处理、用户路径如何组织，这些都可能被代理用默认模式填充。

这会带来一种很隐蔽的错觉：系统看起来在快速前进，实际可能在快速固化未经批准的选择。对普通建造者来说，速度本身没有意义；有意义的是朝着正确产品方向的速度。人类在环的作用，正是让方向检查发生在还来得及修改的时候。

因此，本章的结论很朴素：Agent loop 是一种反馈系统，魔法词这个理解会误导使用者。它能把代理、输出和反馈连接起来，也会把规格说明里的空白放大成实际产品决策。下一章要处理的，就是这些空白如何变成失控点。

1.7 本章小结

- Agent loop（代理闭环）指代理生成输出、接收或评估反馈，再把反馈带入下一步行动的自动化 workflow。
- human-in-the-loop（人类在环）把人类审查、测试、批准、预算控制和产品判断放在循环内部。
- 代理闭环的承诺来自速度、系统化和规模化；它的风险来自人类控制面减少后，代理必须替人做大量隐性产品选择。
- PRD 或 Markdown 规格说明可以降低沟通成本，却很难一次性表达全部产品语境、UX 取舍和架构细节。
- 判断一个闭环是否可靠，要先问反馈信号是否清楚、范围是否有限、停止规则是否存在。

2 全自动 Loop 的失控点：假设、Token 与产品愿景

2.1 聪明开发者类比：失控从规格说明的空白开始

第 2 章把 Agent loop 的风险讲得更具体。Ras Mic 使用了一个创业团队类比：Greg 和 Ras Mic 正在创业，他们雇了一位很聪明的开发者，给出应用目标和功能清单，然后让这位开发者一路把完整产品做出来。开发者很聪明，执行力很强，最终仍然会在大量细节上自行判断。

这些判断包括产品外观、用户体验、信息架构、技术架构、错误状态、数据模型、页面层级、默认值和发布路径。任何真实产品都会遇到这种问题：初始 brief 写不完全部细节，团队成员之间也很难把想法完全对齐。开发者只能补齐空白；代理也会补齐空白。

这个类比的锋利之处在于，它把“AI 代理失控”从神秘能力问题拉回到普通产品开发问题。代理没有读心能力。人类给出的 PRD 或 Markdown spec 越含糊，代理需要自行填补的面积越大；填补面积越大，偏离产品愿景的概率越高。

不完整规格说明会制造隐性假设

把一个含糊产品 brief 交给聪明开发者，然后要求对方独立交付完整产品，结果很可能是一套看似完成、实际充满未经创始人批准决策的系统。Agent loop 也一样：PRD 或 Markdown 规格说明里的空白，会被代理用默认假设填满。

这些假设可能落在 UI、UX、架构、状态管理、权限、文案、数据结构和发布策略上。问题到最后才暴露时，团队面对的已经是一堆互相依赖的实现选择，修复成本远高于一两个局部小错误。

这个 warningbox 是本章的核心。所谓失控，很多时候表现为代理按自己的推断顺利完成了大量工作；系统崩溃只是较少见的极端形态。顺利完成与符合产品愿景之间存在距离，距离越晚被发现，返工越贵。

2.2 PRD 与 Markdown spec 的边界

PRD 的价值在于压缩沟通：目标是什么，功能有哪些，用户是谁，验收标准是什么，技术约束有哪些。Markdown spec 的价值类似，它让代理可以读取结构化要求和上下文。对于小功能、明确脚本、固定页面模板和测试驱动修复，这类文档非常有用。

边界也同样明确。产品愿景包含大量难以提前写死的东西：一个按钮的语气是否合适，首屏承诺是否可信，登录流程是否过早，空状态是否减轻用户挫败感，架构是否服务下个月的商业计划。很多取舍需要看到中间产物后才会出现。文档可以传递目标，无法替代持续判断。

视频里提到服务行业的经验：即便人类团队努力提取客户脑子里的所有想法，也经常会遇到“这里漏了”“我原本想要的是另一种形式”。这说明问题来自表达本身的有限性。AI 代理可以读更多上下文，也仍然会受制于初始表达里的空白。

2.3 隐性假设如何变成产品债务

“假设”听起来像一个很轻的词，实际会变成产品债务。它像代码里的默认参数：当默认值只影响一个局部函数时，改起来很容易；当默认值进入数据库、路由、权限和用户路径后，后续每一步都会围绕它继续生长。

在全自动 loop 里，假设至少会沿着三条线传播。

- **体验线**：代理选择页面布局、交互路径和文案语气，用户感受被默认决策塑造。
- **工程线**：代理选择状态管理、数据模型、API 结构和错误处理，后续功能会依赖这些选择。
- **商业线**：代理决定哪些功能先做、哪些信息突出、哪些用户动作被鼓励，产品定位被实现细节固化。

这些债务的麻烦之处在于，它们常常一开始看起来很合理。直到创始人看到完整产品，或者用户试用后产生困惑，团队才意识到很多选择没有经过讨论。人类在环的意义，就是把这种发现提前到着陆页、认证、后端、核心流程这些阶段性节点。

³视频画面时间区间：00:06:25–00:06:42。

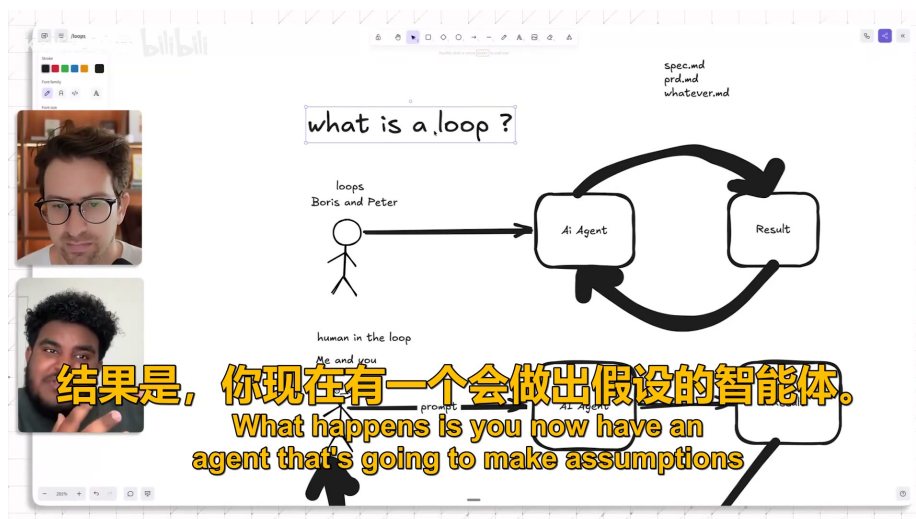


图 3: 规格说明留下空白时，代理会在循环中自动填入假设。³

2.4 Token 预算：循环每跑一轮都会收费

Ras Mic 对 Token 成本的提醒非常 blunt。Token 在这里可以理解为模型使用量和边际成本的代理变量：提示、上下文、代码、反馈、日志、修复建议都会消耗 Token。一个开放式 loop 越会自我迭代，消耗越容易从“几次调用”变成“长时间后台燃烧”。

这种 Token 风险在画面中被讲得很直白：循环听起来很酷，持续运行时也会持续烧钱。

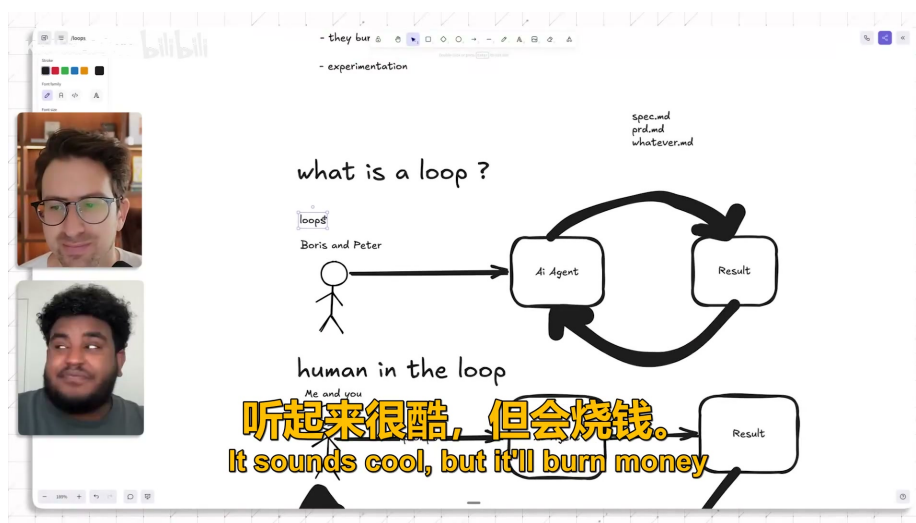


图 4: 开放式循环会持续消耗 Token；预算约束必须进入设计。⁴

这个预算约束可以用一个短式子表达：

$$T \leq B$$

- T : 某个 Agent loop 在一次任务里消耗的 Token 总量，也代表模型调用成本和机会成本的粗略代理变量。
- B : 团队愿意为该任务投入的 Token 预算，包括金钱预算、等待时间和注意力预算。

⁴视频画面时间区间：00:11:32–00:11:40。

这条式子没有复杂数学含义，它只是在提醒普通建造者：循环的边际成本必须小于预算。高预算团队可以把长时间自我修复当作研发投入；普通建造者需要在循环启动前设置范围、上限和人工检查点。

视频里把 Boris 和 Peter 描述为拥有更大 Token 空间的探索者，这应按 speaker claim 处理。这个说法的教学价值，不在于验证某个人究竟花了多少钱，而在于说明资源环境不同会改变工程策略。研究团队可以用大量预算探索未来能力，普通创业者必须对当下产品和账单负责。

2.5 /goal 与 /loop：命令名不同，风险结构相同

Greg 追问 /goal 与 loop 的关系，Ras Mic 的回答很清楚：这些命令在不同工具里名字可能不同，例如 /goal、/loop 或其他 slash command；高层机制都是让用户给出目标、附上文档，然后让代理持续执行直到完成。

这类命令的吸引力很强，因为它把复杂工作包装成一个简单入口。用户输入目标，附上 PRD 或 Markdown spec，系统开始行动。界面越顺滑，越容易让人忽略背后的关键问题：反馈信号来自哪里，停止条件是什么，谁检查中间结果，谁承担错误成本。

因此，/goal 和 /loop 更适合被理解成“自动化入口”，替代产品判断这个定位会制造风险。入口越强大，边界越需要清楚。把一个模糊产品愿景交给自动化入口，系统会把模糊性转化成实现细节；把一个明确、可测、范围有限的任务交给入口，系统才有机会稳定地产出价值。

2.6 产品愿景无法完全写进一次性文档

视频中有一个很实在的产品例子：趋势会变化，视觉偏好会变化，团队今天觉得某种“液态玻璃”风格很好，明天可能又想调整比例、质感或呈现方式。这类判断很难在初始文档里写到足够准确，因为团队自己的想法也在中途变化。

这属于产品探索的正常状态，文档质量只是其中一个变量。产品愿景由用户反馈、创始人品味、市场变化、竞争态势、技术约束和团队资源共同塑造。一次性 PRD 只能捕捉某个时刻的理解，无法承载构建过程中出现的新信息。

对服务公司来说，这个问题尤其熟悉。团队会尽力访谈客户、抽取需求、整理规格说明，最后仍会遇到客户说“这里和预期不同”“这个交互应该换一种方式”。如果人类之间都难以完全同步产品想法，代理在同样模糊的输入下也会产生偏差。

2.7 普通建造者需要的是有上限的自动化

本章的工程结论很实际：普通建造者可以使用 Agent loop，但需要给它设定上限。上限包括任务范围、Token 预算、迭代次数、可验收输出和人工检查点。没有上限的自动化，会把不确定性、预算消耗和返工成本一起放大。

更稳妥的做法是把大目标拆成可检查的小单元。例如，先让代理生成着陆页，再由人检查定位和转化路径；再让代理处理认证，并用浏览器手动验收关键路径；再让代理推进后端，同时用测试和代码审查作为反馈。这样做会增加一些摩擦，却能让错误在早期显形。

这也解释了后续章节的方向。第 3 章会讨论哪些实验和原型可以适当放手；第 4 章会给出更可落地的 Greptile 代码审查闭环。第 2 章的任务只是把危险讲清楚：完整文档不会消除产品假设，slash command 不会自动提供可靠反馈，Token 预算会真实消耗。

这里的重点可以进一步压缩成一个实践规则：自动化入口负责速度，人类检查点负责方向。

2.8 本章小结

- 全自动 loop 的主要失控点来自规格说明里的空白；代理会用隐性假设补齐这些空白。
- UI、UX、架构、状态管理、权限、数据结构和发布策略都可能被代理自动决定，最后变成产品债务。
- PRD 或 Markdown spec 能降低沟通成本，无法穷尽产品愿景和构建过程中出现的新判断。
- Token 是模型使用预算和边际成本的代理变量；开放式循环每跑一轮都会继续消耗预算。
- /goal、/loop 和类似 slash command 是自动化入口，可靠性取决于反馈信号、停止条件和人工检查点。
- 普通建造者应使用有上限的自动化：小范围、明确验收、预算封顶、阶段性人工检查。

3 什么时候可以放手：实验、原型与有限目标

3.1 实验型任务的边界

第 3 章承接上一章的警告：Agent loop 不能被一概视为糟糕工具。Ras Mic 给出的更精确判断是，loop 适合实验、原型和低风险小工具，尤其适合那些“细节不重要，先跑起来最重要”的任务。这个边界很关键，因为它把问题从“要不要用代理循环”改成“这个任务是否允许代理自行填补细节”。

实验型任务有三个特征。第一，输出只需要服务一个短期目的，例如演示、测试想法、构造内部 benchmark。第二，细节偏差带来的损失较小，例如页面不够漂亮、交互不够顺、实现方式偏粗糙。第三，使用者很快会丢弃或重写它，系统不会立即进入生产路径。满足这三个条件时，代理循环的价值来自速度：它把一个想法迅速变成可运行样品。

这和前一章讨论的产品愿景形成鲜明对照。产品愿景会把 UI、UX、架构、定位和商业取舍绑在一起；实验工具通常只要求“能用”。对于“能用即可”的场景，代理补齐细节的风险明显降低。

3.2 模型说谎模拟器：细节不敏感时的正例

Ras Mic 举了一个自己的例子：他准备做演讲时，想为 AI 模型创建一个模拟器，用来评估模型在游戏式场景中的说谎能力。字幕里提到这个场景有反派角色、冒名顶替者，以及一群人试图找出真凶。这类模拟器更像内部研究工具，目标是跑通 benchmark 思路，不承担真实用户发布压力。

他给代理的要求可以概括为：需要一个模拟，需要一个 benchmark，需要马上执行。这里没有要求精致品牌、复杂登录、商业转化路径，也没有强调视觉设计。Ras Mic 明确说自己不在意外观和细节，只希望模拟基准能够运行。于是，loop 在大约一个半小时内帮他完成了一个可用结果。

实验型 loop 的价值

实验型 loop 的价值来自“快速把问题外化”。当使用者脑中只有一个粗略想法时，代理可以先生成一个可运行版本，让人看到问题长什么样。这个版本可能粗糙，仍然有价值，因为它把抽象想法变成了可测试对象。

这类任务像白板草图：重点是暴露结构，完成装修不在核心目标里。只要使用者清楚自己会继续筛选、删改或重写，代理填补细节的风险就可控。

这个例子的教学价值很直接：如果任务目标是验证一个小想法，允许粗糙实现，且不会马上影响用户和收入，Agent loop 可以明显提高探索速度。相反，如果任务直接进入产品核心路径，粗糙实现会变成后续债务。

3.3 “复制酱料”和“创造酱料”的区别

视频中有一句很重要的判断：AI 可以复制 sauce，却很难创造 sauce。这里的 sauce 可以理解为产品里的独特味道：定位、体验、审美、节奏、用词、交互细节和商业直觉。它指向一组长期积累的判断，范围远大于单个功能或一段代码。

复制 sauce 指的是已有风格已经足够清楚，代理只需要照着样式扩展。例如已有十个页面，现在要生成第十一个相似页面；已有固定测试模式，现在要补充更多 case；已有代码审查规则，现在要把反馈转化为修复。创造 sauce 指的是标准本身还没有稳定下来：产品要给谁用，页面应该让用户感到什么，哪个功能应该先做，哪些复杂度应该先砍掉。

两者的差别可以用三个普通问题判断：任务约束是否清楚，反馈是否稳定，错误是否容易撤回。模型说谎模拟器就属于这种场景：约束大致清楚，反馈可以通过运行模拟获得，错误也容易丢弃。

3.4 老虎机类比：开放循环为什么会让人上头

Greg 把这些循环形容成老虎机。这个类比很准确，因为开放式 Agent loop 会制造一种“下一轮也许就好了”的心理预期。用户看到代理一直在工作、一直在修改、一直在输出，就容易继续投入 Token 和时间，希望下一次迭代给出更好的结果。

老虎机的危险在于反馈间歇性出现。偶尔一次结果不错，会让人忘记前面很多次的成本。Agent loop 也类似：某一轮突然修好了一个 bug，或者生成了一个看起来很完整的页面，用户就容易忽略之前消耗的上下文、订阅额度、等待时间和注意力。

老虎机式循环的风险

开放式 loop 最容易诱导用户持续追加成本。每一轮都看起来离目标更近，实际可能只是在同一批隐性假设周围打转。普通建造者需要把循环设计成有预算、有次数、有验收标准的流程，而不能把“继续再试一轮”当作默认策略。

这也是 Token 预算反复出现的原因。Token 对应金钱、等待时间和机会成本。字幕里提到 20、100 或 200 美元级别的订阅预算；这种普通订阅环境，和前沿团队的大规模实验预算完全是两种工程环境。

3.5 强团队的 loop 为什么更容易成功

Ras Mic 随后提到 Boris 和 Peter 这类更强团队可能拥有更完整的测试体系。他说这部分带有猜测性质，因此笔记也应按 speaker claim 处理。核心意思是：他们的 loop 可能已经接入了测试套件、浏览器操作、截图检查和更强的质量保障框架；“代理自由发挥”只是外部观察者容易误读出的表面动作。

这点很重要。普通用户看到“某些高手可以让代理跑很久并成功交付”，容易只复制表面动作：启动 loop，让代理继续。但真正让闭环变强的通常是反馈基础设施。测试套件会告诉代理代码是否通过；浏览器自动化会告诉代理页面是否真的可见；截图会暴露布局错位；日志会暴露运行错误；预算和迭代上限会阻止无限燃烧。

可以把强团队的 loop 理解为带护栏的自动驾驶：它仍然在自动执行，周围却有传感器、路线、仪表盘和接管机制。普通无护栏 loop 更像只给出目的地，然后希望系统自己猜出全部道路规则。

强 loop 的隐藏组件

一个可靠 loop 往往需要这些组件：自动测试、浏览器验收、截图或视觉比较、错误日志、评分器、预算上限、迭代上限、人工接管点、回滚策略。缺少这些组件时，loop 只是把“继续生成”包装成自动化。

3.6 Peter 的 Token 成本只能作为说话者轶事

字幕结尾提到 Peter 一个月烧掉价值 130 万美元的 Token。这个数字在笔记中只能作为视频说话者的轶事引用，不能被当成独立核验后的事实。它的作用是说明数量级差异：前沿探索者可以把大规模 Token 消耗看成研发成本，普通建造者通常没有这种条件。

因此，更稳的读法是：不同资源环境对应不同策略。研究团队也许需要用大量试验探索未来能力；普通产品团队更需要把每一次 loop 放进预算、验收和用户反馈框架里。

3.7 本章小结

- Agent loop 适合实验、原型、小工具和低风险 benchmark，尤其适合细节不敏感的任务。
- 模型说谎模拟器说明了一个正例：目标明确、细节可粗糙、错误可丢弃时，loop 可以快速把想法变成可运行对象。
- “复制酱料”适合自动化扩展；“创造酱料”需要人类判断、用户反馈和产品品味。
- 开放式 loop 很像老虎机，会诱导用户持续追加 Token 和时间，因此需要预算和迭代上限。
- 强团队的 loop 成功，往往依赖测试、浏览器验收、截图、日志、评分器和接管机制这些隐藏基础设施。
- Peter 的高额 Token 成本应按视频中的说话者轶事处理，它提醒读者关注资源环境差异。

4 可落地的闭环：Cursor、GitHub 与 Greptile 代码审查

4.1 代码审查是一个更适合闭环的场景

第 4 章给出全片最具体的可执行样板。Ras Mic 说自己的代码审查流程就是一个 loop：他主要使用 Cursor 写代码或修改代码，用 GitHub 做版本控制和 pull request 承载，用 Greptile 作为代码审查代理。字幕里的“reptile”“grep p”等写法是 ASR 错误，结合语境应统一为 Greptile。

这个例子有教学价值，因为它不依赖模糊的产品感觉。代码审查至少有一组可描述的反馈：安全问题、边缘情况、遗漏逻辑、测试失败、风格问题、评分。反馈越结构化，loop 越容易知道下一步该修什么。

⁵视频画面时间区间：00:13:02–00:13:31。



图 5: Ras Mic 描述的基础链路：Cursor 写代码，GitHub 承载版本控制与代码审查。⁵

4.2 闭环结构：生成、审查、修复、再推送

这个流程可以拆成四步。

1. Cursor 生成或修改代码。
2. 开发者把代码推送到 GitHub。
3. Greptile 读取 pull request，并给出审查意见与评分。
4. Cursor 读取 Greptile 的审查意见，继续修改代码，再推送到 GitHub。

这就是一个典型反馈闭环：上一轮代码进入审查，审查意见进入 Cursor，Cursor 再把修复推回 GitHub。关键在于，代理下一轮的输入来自上一轮审查反馈。反馈越具体，下一轮修复越容易朝正确方向移动。

4.3 五分制评分把停止条件显性化

Ras Mic 最喜欢 Greptile 的一点，是它会给出五分制评分。他的规则是：评分需要超过 4/5 才允许进入生产环境；如果只有 2/5 或 3/5，代码必须被继续审查和修复。这里的重点在于，评分把“够不够好”变成了一个可操作门槛。

这个门槛可以用大纲里的符号写成：

$$s \geq s_{\min}, \quad s_{\min} \approx 4/5$$

其中 s 是 Greptile 给出的代码审查分数， $s_{\min}$ 是最低可接受发布阈值。真实团队当然还需要测试、人工审查和部署策略；这个公式只表达视频里的核心思想：闭环必须有可见阈值。

⁵视频画面时间区间：00:14:08–00:14:35。

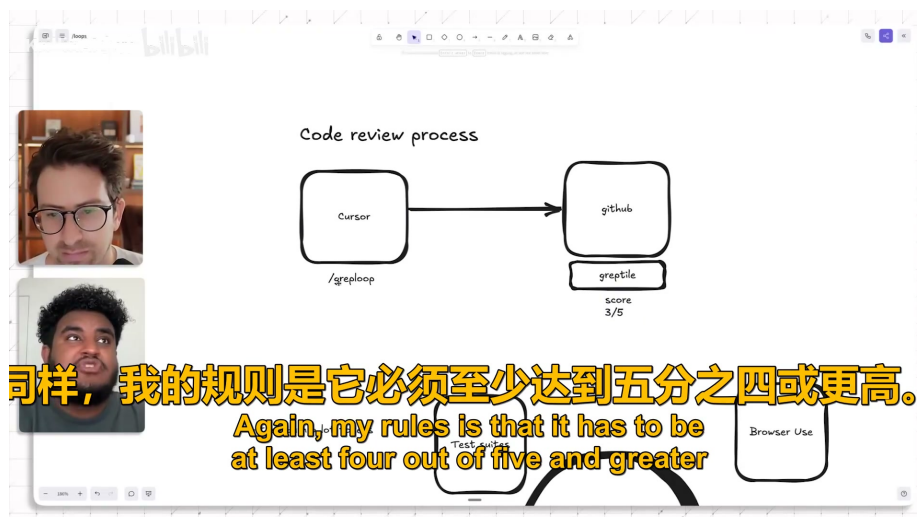


图 6: 五分制评分把发布门槛显性化：低于阈值时继续修复。⁶

Greptile 代码审查 loop 的有效边界

这个 loop 成立的前提，是 Greptile 给出明确反馈和五分制评分。Ras Mic 的规则可以压缩成三点：低于大约 4/5 时继续修复；理想目标是 5/5；如果多轮修复仍达不到目标，大约五轮后停止，让人介入判断。评分门槛、目标分数和迭代上限共同构成这个闭环的边界。

4.4 五轮上限：防止循环无限燃烧

视频里还有一个细节：如果评分没有达到理想状态，loop 会继续采纳反馈、推送 GitHub、等待新审查；它有大约五轮的上限，或者直到达到 5/5 才停止。这个上限非常重要，因为它把自动化从“无限尝试”变成“有限搜索”。

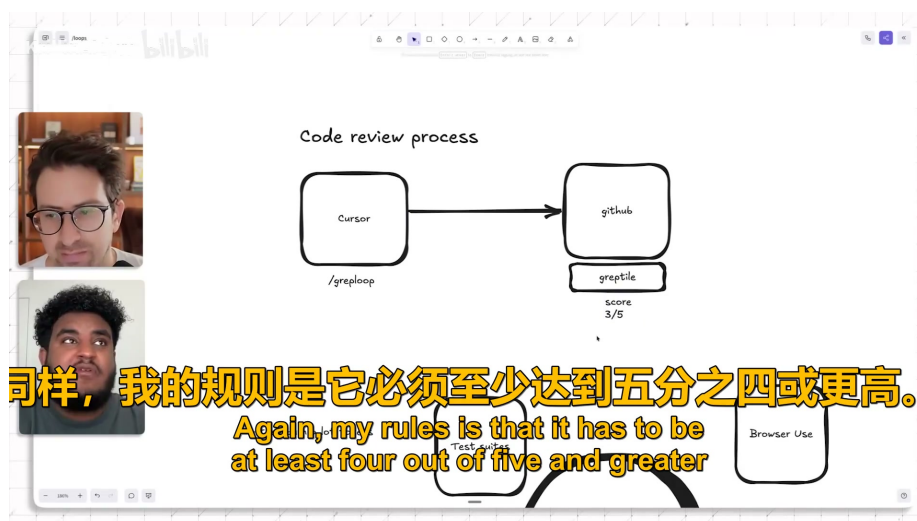


图 7: 评分、修复、再评分构成封闭循环；迭代次数必须有上限。⁷

一个实用退出规则可以写成：

⁷ 视频画面时间区间：00:14:55–00:15:49。

stop if $s = s_{\text{target}}$ or $i \geq i_{\text{max}}$

- s 表示当前审查分数。
- s_{target} 表示理想停止分数，视频里对应 5/5。
- i 表示已经完成的循环轮数。
- i_{max} 表示最大重试次数，视频里约为五轮。

这个规则背后的工程直觉是：如果五轮后仍然无法达到目标，问题可能已经超出代理独立修复范围。此时继续自动运行的收益下降，人工检查的价值上升。

4.5 为什么这是封闭循环

Ras Mic 强调，这个流程有意义，因为它是非常封闭、目标明确的循环。所谓封闭，指反馈来源固定、任务范围固定、动作集合固定、目标分数固定。Greptile 负责指出问题并打分；Cursor 负责阅读评审并修复；GitHub 负责承载代码状态和审查上下文。

这类闭环和“让代理构建完整应用”差别很大。完整应用构建需要大量开放判断：用户是谁，页面语气是什么，哪些功能要砍掉，架构如何服务未来路线。代码审查的反馈面更窄，模型更容易把反馈转化为具体修复。

封闭循环的判断标准

封闭循环至少满足四个条件：输入范围清楚，反馈来源稳定，质量标准可表达，停止条件可执行。代码审查、测试修复、格式转换、批量模板生成通常更接近这种结构。开放式产品构建缺少稳定反馈面，因此需要人类在环。

这个定义也解释了为什么同样叫 loop，实际风险差距很大。一个有评分器、有版本控制、有迭代上限的 loop，和一个从模糊 PRD 直接生成完整产品的 loop，工程性质完全不同。

4.6 一千行限制：固定生态里也会崩

Ras Mic 并没有把自己的流程说成完美。他明确指出，这个 loop 有时会崩，典型场景是一次推送超过大约一千行代码。代码变更多到一定程度后，审查代理很难完整理解上下文，也很难给出高质量反馈。结果就是评分上不去，修复方向也会变得混乱。

这个限制很现实。代码审查代理的上下文窗口、检索能力、问题排序能力和反馈质量都有边界。大 diff 会把多个问题混在一起：安全、架构、重构、样式、边缘情况、测试缺失。代理可能抓住一部分问题，同时漏掉更关键的问题。

大 diff 会击穿封闭反馈

即使在 Greptile 这种固定反馈循环里，过大的 pull request 也会让审查质量下降。Ras Mic 给出的经验线是大约一千行变更：超过这个量级后，审查代理很难完整理解上下文，也更难给出稳定高分。处理方式是把大变更拆成多个小 pull request，让每次审查面对更窄的代码面。

因此，Ras Mic 的处理方式是把大变更拆成多个部分，多次推送，让 Greptile 分别审查。这条建议也适用于人类团队：小 pull request 更容易审查，更容易回滚，更容易定位错误。

⁸视频画面时间区间：00:17:01–00:17:36。

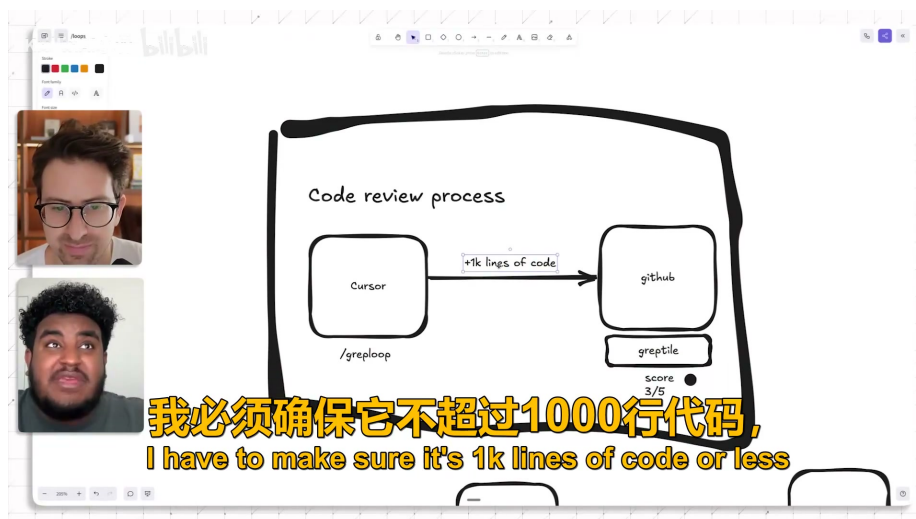


图 8: 超过约一千行的巨大变更, 会让代码审查闭环明显变差。⁸

4.7 代码审查样板带来的通用启发

这个样板的通用启发有三条。

- **把反馈外部化**: 让反馈来自测试、审查代理、浏览器验收或用户试用, 避免代理自我感觉良好。
- **把门槛数字化**: 例如 4/5 发布门槛、五轮迭代上限、一千行 diff 上限。
- **把任务切小**: 每轮变更越小, 反馈越准确, 修复越可控。

这三条把 Agent loop 从魔法叙事拉回工程流程。好的 loop 需要清楚的反馈器、清楚的边界和清楚的退出条件, 不能让代理永远自己想办法。

4.8 本章小结

- Cursor + GitHub + Greptile 是视频中最具体的可落地 Agent loop 样板。
- 代码审查适合闭环, 因为反馈信号较明确: 安全问题、边缘情况、遗漏逻辑、测试、评分。
- Greptile 的五分制评分让发布门槛显性化, 例如 $s \geq s_{\min}$ 才考虑发布, 视频语境里的 $s_{\min} \approx 4/5$ 。
- 五轮上限和 5/5 目标把循环限制为有限搜索, 避免无限消耗 Token。
- 超过约一千行的大 diff 会削弱审查质量, 因此应拆成多个小 pull request。
- 这个样板的核心是反馈器、版本控制、评分阈值、迭代上限和小步变更; 工具名本身只是实现载体。

5 边界条件: 二元反馈、人类反馈与创造性工作

5.1 从代码审查推广到边界判断

前一章的 Greptile 代码审查流程给出了一个可操作样板: Cursor 生成或修复代码, GitHub 承载 pull request, Greptile 给出审查意见和五分制评分, Cursor 再把这些反馈带回下一轮修改。这个

流程之所以成立，关键在于它有清楚的反馈信号、有限的作用范围，以及明确的停止条件。

Ras Mic 在结尾阶段把这个样板推广成一条判断边界：代理闭环有用的位置，通常出现在结果可以评分、可以验收、可以重复生成的任务里。结果越接近开放式产品判断，闭环越需要人类参与。这个判断很直接，也很残酷：很多普通建造者以为自己需要一个全自动的 `/goal` 或 `/loop`，实际缺的往往是中途看到产品、让用户试用、让创始人重新判断方向的机会。

这里的核心问题可以压缩成一句工程提问：反馈信号由谁给出，错误又由谁买单？如果反馈信号来自测试、评分器、模板规则或明确的人工验收，Agent loop（代理闭环）可以节省重复劳动；如果反馈信号来自市场感觉、用户反应、产品品味和商业取舍，human-in-the-loop（人类在环）就成了控制风险的主路径。

5.2 二元反馈：循环最容易闭合的地方

二元反馈指的是输出可以被清楚判定为通过或未通过，或者至少能被稳定地打分。代码审查就是这种任务：有没有安全问题、有没有边缘情况错误、测试是否通过、评分是否达到 4/5，这些信号虽然也会有误差，却足够结构化。SEO 页面生成也是类似场景：给定关键词、标题模板、段落结构、内部链接规则和元信息格式后，系统可以批量生成页面，再用规则或审核器检查是否符合模板。

二元反馈 vs 创造性反馈

二元反馈适合闭环，因为它像门禁：通过、失败、得分高、得分低，系统可以据此进入下一步。代码审查、SEO 页面生成、固定模板内容、重复性数据整理，都可以建立这种反馈面。创造性反馈更像产品评审会：用户是否理解这个页面，创始人是否认可定位，品牌语气是否合适，交互是否让人愿意继续使用，架构选择是否服务商业节奏。这类反馈需要用户、创始人、设计者和预算负责人在过程中介入。对于初创产品、创意 UX、面向用户的应用，闭环里缺少人的判断，就会把一连串未经批准的假设固化成“完成品”。

这也是 Ras Mic 提到 SEO 和页面生成时给出的边界：如果某个团队已经有固定 SEO 公式，希望生成三百个看起来和听起来高度相似的页面，代理闭环可以承担大量机械工作。这样的任务像批量印刷：版式固定、内容槽位固定、验收规则固定，机器多跑几轮的价值很高。

不过，SEO 批量生成仍然需要最后的人工抽检。作为笔记的实践延伸，事实错误、品牌风险、重复内容质量、法律或合规措辞都可能被模板评分漏掉。闭环负责规模化，人类负责边界和责任。

5.3 SEO 页面生成为什么更适合循环

用 SEO 页面生成来理解闭环，很适合普通建造者。假设团队要为三百个长尾关键词生成落地页，每个页面都遵循同一套公式：

- 标题包含目标关键词和清楚的搜索意图。
- 首段回答用户最可能提出的问题。
- 正文按固定模块展开，例如问题、步骤、示例、FAQ。
- 页面包含元描述、内部链接、结构化数据和一致的行动按钮。
- 审核器检查字数、关键词覆盖、重复率、链接格式和基本事实一致性。

⁹ 视频画面时间区间：00:19:55–00:20:22。

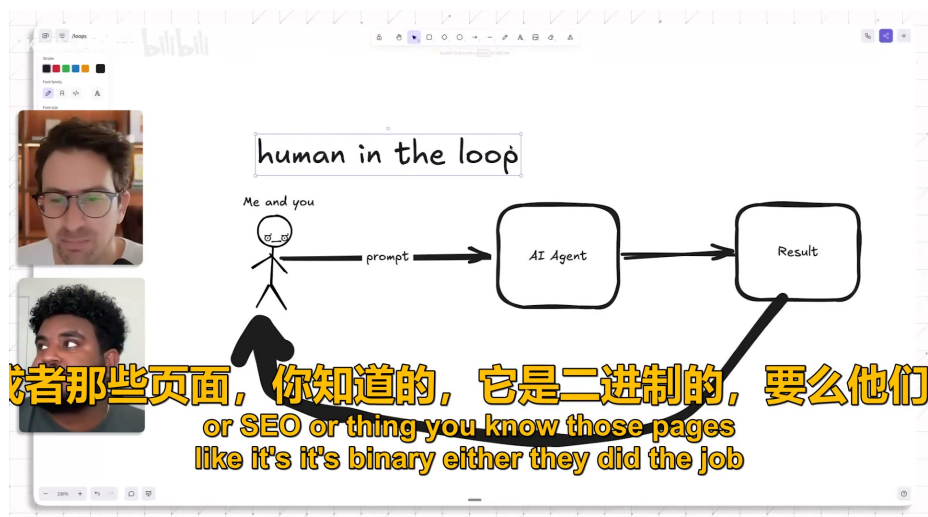


图 9: Ras Mic 把代码审查、SEO 等任务归入更适合闭环的二元反馈空间。⁹

这样的流程天然适合 Agent loop。代理先生成页面，审核器发现缺项，代理按反馈修复，然后继续生成下一批。反馈信号有足够的硬边界，错误的代价也较容易通过抽检和回滚控制。

这类任务的教学价值在于：闭环真正擅长的是把“已知标准”变成“持续产出”。它对“发现标准”帮助有限。SEO 模板已经由人定好了，代理只是把模板执行到更大的规模。对于正在探索产品定位的团队，标准本身还在变化，闭环就会过早地把未成熟标准放大。

5.4 创造性产品工作为什么需要中途用户反馈

视频里最重要的边界提醒来自面向初创产品的场景。播客听众想做应用、网站、SaaS 或微 SaaS，真正要解决的通常是产品与市场的匹配问题。这个问题需要在构建过程中不断展示中间版本：给潜在用户看，让他们试，让他们困惑，让他们指出哪里没价值，让创始人重新排序功能。

如果系统从一个 PRD 或 Markdown 规格说明出发，一路自动构建到“完成”，中途缺少用户反馈，风险会沿着每一层决策累积。第一层是假设用户要什么，第二层是假设界面如何呈现，第三层是假设交互路径怎样安排，第四层是假设技术架构怎样支撑后续变化。到最后，团队面对的是一个看似完整、实际充满隐性选择的产品。

创造性产品工作还有一个常被低估的变量：品味。在这里，品味是对用户、语境、节奏、取舍和约束的综合判断。一个注册页按钮放在哪里、默认状态怎么写、首屏价值主张是否可信、空状态是否降低挫败感，这些都很难在初始文档里一次性写准。人类反馈的意义就在这里：它让错误尽早暴露，让方向可以在成本较低时改变。

5.5 全自动驾驶类比：旅程中无法下车

Ras Mic 用 full-autonomous-driving（全自动驾驶）做了一个很形象的类比：从迈阿密一路开到南卡罗来纳州查尔斯顿，用户按下开始后，途中看到路线偏离，或者发现路边有一个想停下来的小餐馆，却无法中途下车。这个类比把开放式闭环的危险讲得很清楚：全程自动化带来的问题包括最终结果对错，也包括途中失去重新选择的权利。

在产品构建里，“无法中途下车”对应三类损失。第一，用户反馈被推迟，团队很晚才知道方向是否有价值。第二，Token 和时间持续消耗，修正窗口越来越窄。第三，早期错误会变成后续实现

¹⁰ 视频画面时间区间：00:18:58–00:19:07。

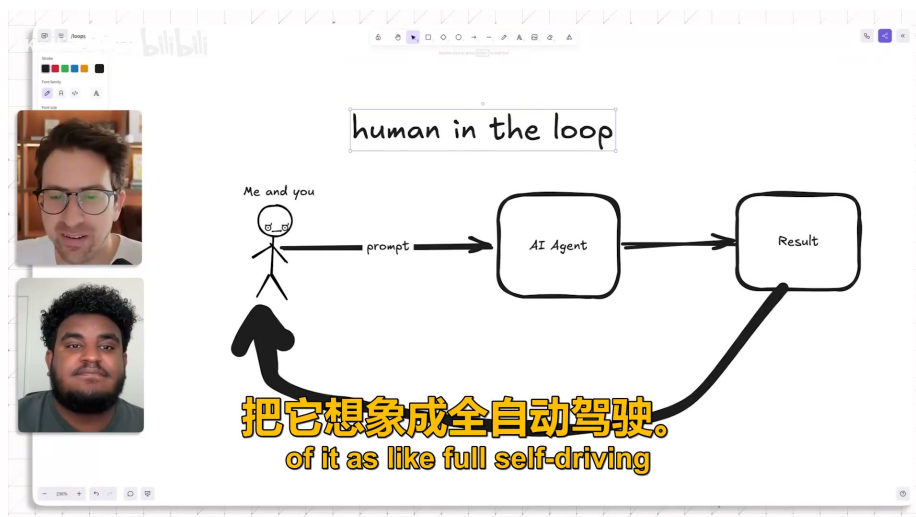


图 10: 全自动驾驶类比说明了中途无法接管的风险。¹⁰

的依赖，后面每一步都在为前面的假设补洞。

延迟校正的代价

开放式代理闭环最大的风险，是把纠错点推到太晚。一次错误的 UI 假设，可能带来错误的数据模型；错误的数据模型，又会影响权限、路由、状态管理和部署路径。越晚发现，修改成本越高。对于创造性产品，最便宜的反馈通常来自中途演示、用户试用和创始人判断。

这个类比也提示了一个隐藏前提：真正可靠的自动驾驶需要传感器、地图、路线规划、实时决策、接管机制和责任体系。把这套比喻放回 Agent loop，普通建造者也需要类似组件：测试、评分器、浏览器检查、日志、预算上限、人工接管点和回滚策略。缺少这些控制面，开放式闭环更像把车开出去之后等待运气。

5.6 未来 caveat：高预算实验与普通建造者的距离

视频没有把未来完全关上。Ras Mic 承认，百分之百自动化在未来某个阶段可能会变得可行。前沿团队有理由探索自我修复系统、测试套件、浏览器自动化和更复杂的反馈架构。对这些团队来说，Token 预算、实验成本和失败成本属于研发投入的一部分。

普通建造者面对的是另一套约束。二十美元、一百美元或二百美元级别的订阅预算，很难支撑长时间开放式循环；更重要的是，普通建造者的产品问题往往没有完备测试集，市场反馈也无法由模型自己伪造出来。高预算实验室可以为了未来能力支付探索成本，普通创业者需要为当下产品结果负责。

因此，未来 caveat 的正确读法是：全自动闭环可能会进步，当前使用时仍要按任务边界和反馈质量做决策。把未来能力提前套用到今天的初创产品构建，会让团队把钱、时间和注意力花在错误的自动化层级上。

5.7 human-in-the-loop 的最终判断

这一段最后收束到 human-in-the-loop（人类在环）。人类在环包含的远多于“最后看一眼”：定义目标、检查中间结果、吸收用户反馈、批准最终发布。对代码审查来说，人类可以把 Greptile 的

评分当成重要信号；对产品工作来说，人类必须把用户反应、商业节奏和产品品味带回系统。

更实用的工作流通常长这样：让代理处理局部任务，让自动反馈处理机械验收，让人类在关键节点判断方向。比如一个 SaaS 原型可以让代理生成注册、计费、仪表盘和设置页的初稿；每完成一个可演示切片，就让创始人、用户或设计负责人检查。这种节奏让自动化承担速度，让人类承担判断。

5.8 本章小结

- Agent loop 适合反馈明确的任务，例如代码审查、SEO 页面生成、固定模板内容和有限目标的实验。
- 二元反馈或评分反馈能让循环知道下一步该修什么，也能定义何时停止。
- 创造性产品工作需要中途用户反馈，因为定位、UX、品味和商业取舍无法在初始规格说明中一次性写完。
- full-autonomous-driving 类比说明了开放式闭环的真实风险：旅程开始后，团队可能很晚才发现路线已经偏离。
- 未来全自动化可能进步，当前普通建造者仍应把 human-in-the-loop 作为默认控制结构。

6 总结与延伸：最好的 Loop 是有约束的人机协作

6.1 收束：Ras Mic 的最终判断

结尾阶段，Greg Isenberg 感谢 Ras Mic 把 Agent loop 讲清楚，并把常见兴奋情绪拉回现实。真正有信息量的收束只有一句：人类参与才是当前最好的循环。这里的“最好”指向工程约束下的现实选择：当反馈信号可靠、范围受限、停止条件明确时，代理闭环能显著提高效率；当任务牵涉产品愿景、用户感受、创意取舍和预算责任时，人类必须留在系统里。

这条结论也解释了整期视频的结构。前半段先定义 Agent loop（代理闭环），再说明 PRD 或 Markdown 规格说明无法覆盖全部产品细节；中段给出 Greptile 代码审查这个可落地样板；最后把样板抽象成边界条件。换句话说，视频讨论的重点落在一种判断方法上：什么任务能被闭环，什么任务需要人类中途介入。

6.2 把定义压缩成一条公式

如果把全片压缩成一个工程定义，可以写成：

$$\text{Agent loop} = \text{agent action} + \text{feedback signal} + \text{next action} + \text{stop rule}$$

- **agent action**：代理执行一次具体动作，例如生成代码、修复 pull request、生成 SEO 页面或运行实验。
- **feedback signal**：系统得到反馈，例如测试结果、审查评分、模板规则、人工评论或用户试用反馈。
- **next action**：代理把反馈带回下一步，继续修复、重写、缩小范围或提交新版本。

- **stop rule**: 系统知道何时退出，例如达到目标评分、超过迭代上限、预算耗尽或人工批准发布。

这条公式的关键在第二项。没有可靠反馈信号的循环，会退化成自动化猜测；没有停止规则的循环，会把 Token、时间和注意力持续投入一个越来越难解释的过程。真正可用的闭环必须同时回答两个问题：系统凭什么知道自己变好了，系统什么时候停下来？

实用决策规则

把 Agent loop 用在范围有限、反馈可测、错误可回滚的任务上；把人工检查点保留在模糊、创造性、产品定义型工作里。只要任务需要用户反应、创始人品味、商业取舍或最终责任判断，human-in-the-loop（人类在环）就应成为默认结构。

6.3 核心取舍：自治、省力与延迟纠错

Agent loop 的诱惑在于减少手动提示。人类给出一个目标，代理自己生成、检查、修复、再提交。这个想法听起来像把生产力拉满，尤其适合重复性强、验收信号明确的工作。

自治程度上升以后，隐藏成本也会上升。第一，代理会填补规格说明里的空白，很多填补动作都是未经批准的产品假设。第二，循环每跑一轮都会消耗 Token，预算会随着尝试次数增长。第三，纠错点被推迟，错误会在多轮实现里叠加。用更直观的话说，自治减少了打字次数，也增加了“系统自己做主”的面积。

人类参与会增加摩擦：要看中间版本，要做评审，要决定下一步，要承担发布责任。这个摩擦有实际价值。它把品味、预算控制、用户反馈和商业判断放回流程里，让系统在方向偏离时更早停下。对普通建造者来说，这种摩擦常常比全自动带来的速度更值钱。

6.4 适用性矩阵

下面的矩阵把全片判断压缩成一个可执行表。它是一条基于当前视频语境和当前工具能力给出的实用分界。

这个表格相当于把视频里的判断压缩为一张工程检查表：闭环越依赖客观反馈，适配度越高；闭环越依赖品味和用户判断，越需要人类留在流程中。

维度	强适配 Agent loop	弱适配 Agent loop
反馈信号	测试通过、审查评分、模板规则、benchmark 分数	用户是否喜欢、定位是否清楚、体验是否有吸引力
典型任务	代码审查、测试驱动修复、固定模板 SEO 页面生成、模拟、benchmark、快速原型	新产品 UX、初创公司定位、创意应用设计、宽泛架构选择、需要中途用户反馈的工作
范围	小 PR、小模块、固定页面族、有限实验	多功能产品、开放式应用、端到端商业系统
停止条件	达到 5/5、通过测试、超过迭代上限、预算封顶	“感觉差不多”、等待最终大验收、用户反馈被推迟到末尾
风险控制	可回滚、可抽检、可重跑、可拆分	隐性假设叠加，方向错误后修改成本高

这个矩阵最重要的用法，是在启动循环前逼迫团队说清楚反馈信号。只要团队无法写出验收标准，代理闭环就会把模糊目标放大。对于产品工作，正确姿势通常是把大目标切成可演示的小切片，每个切片设置人类评审和用户反馈点。

6.5 Greptile loop 作为具体模型

Greptile 代码审查流程是全片最具体的正例。它由四个窄环节组成：Cursor 负责写代码或修复代码；GitHub 接收 pull request；Greptile 审查并打分；Cursor 再读取意见、修改并推送。循环退出条件也清楚：达到 5/5，或者触发大约五轮的迭代上限。

这个停止逻辑可以写成：

$$\text{stop if } s = s_{\text{target}} \text{ or } i \geq i_{\text{max}}$$

- s ：Greptile 代码审查代理给出的当前评分，按视频语境是五分制。
- s_{target} ：理想停止分数，视频里对应 5/5。
- i ：当前循环迭代次数。
- i_{max} ：最大重试次数，视频里约为五轮。

这个模型之所以值得保留，是因为它展示了“有约束闭环”的四个零件：任务小、反馈明确、停止条件明确、人工仍拥有生产判断。即使 Greptile 给出高分，发布到生产环境的责任也属于人类团队。评分是信号，发布是判断。

6.6 普通建造者与前沿实验室

视频里反复出现的另一个区分，是普通建造者与前沿实验室的资源差异。高预算团队可以构建昂贵的自我修复系统：测试套件、浏览器自动化、截图检查、长时间 Token 消耗、复杂评估器、失败后的再尝试。这些团队通过大量实验换取未来能力，投入本身就是研发策略的一部分。

普通建造者面对的约束更硬。Token 预算有限，产品细节还在探索，用户反馈无法由模型代替，发布错误会直接影响信任和现金流。对这类团队，理性的设计是窄范围、短循环、明确上限、人工检查。一个简单预算条件可以写成：

$$T \leq B$$

- T ：闭环过程消耗的 Token，可以理解为模型使用量和边际成本的代理变量。
- B ：团队愿意为这个任务投入的 Token 预算。

这个不等式表达的道理很朴素：当循环消耗超过预算，自动化再炫也会变成亏损。更麻烦的是，预算还只是显性成本；方向错误、用户反馈延迟、团队误以为“已经完成”的机会成本，往往更贵。

6.7 未来 caveat 与当下默认

Ras Mic 没有否认未来的全自动能力。他承认，百分之百自动化在未来某个时点可能会变得可行；Boris、Peter 这类高预算探索者也有合理动机去构建自我修复系统。这部分应按视频里的 speaker claim 处理，本文只把它作为资源环境差异的例子。这个 caveat 很重要，因为它避免把当前判断误读成永久否定。

同时，视频给普通建造者的当前建议非常明确：在今天的工具条件和预算条件下，human-in-the-loop 仍然是最稳妥的默认循环。它让 AI 承担生成、修复和重复执行，让人类承担目标、品味、预算、用户反馈和最终发布。好的循环追求把人放在最有杠杆的位置，排斥人类判断会削弱系统的可靠性。

¹¹ 视频画面时间区间：00:21:43–00:21:50。

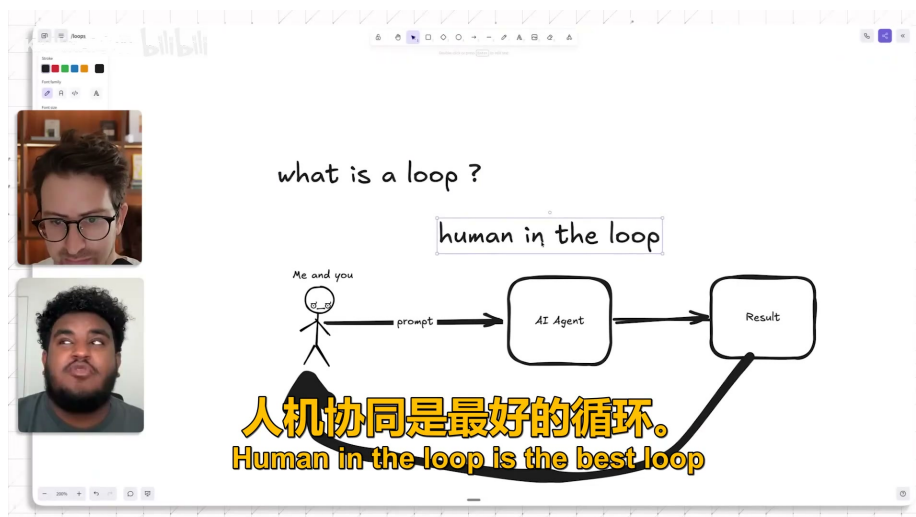


图 11: 结尾的核心判断: human-in-the-loop 仍是当前最可靠的默认闭环。¹¹

6.8 读者检查清单

在把某个任务交给 Agent loop 前, 可以逐条检查:

1. 反馈信号是否客观? 例如测试、评分、模板规则、审查意见或清楚的人工验收标准。
2. 任务范围是否足够小? 例如一个 PR、一个页面族、一个脚本、一个实验, 范围应明显小于整套产品。
3. 是否存在停止条件? 例如达到目标分数、通过测试、最多运行五轮或达到预算上限。
4. Token 预算是否可接受? 循环失败时, 团队是否愿意承担已花掉的成本。
5. 人类或用户能否检查有意义的中间结果? 例如可点击原型、可审查 PR、可读页面草稿或可验证实验输出。
6. 如果任一答案薄弱, 采用有监督工作流, 并设置人工检查点, 避免开放式循环。

6.9 本章小结

全片的最终判断可以浓缩为一句话: 最好的 Loop 是有约束的人机协作。代理负责速度、重复、局部修复和规模化产出; 人类负责目标、反馈、品味、预算和责任。闭环的价值来自清楚边界, 开放式自治的风险来自隐藏假设。普通建造者最该学习的是如何设计一个反馈信号可靠、停止条件明确、人类判断及时进入的工作系统。